

Wydział Informatyki Politechniki Białostockiej Przedmiot: Systemy operacyjne	Data: 18.03.2026r.
Temat: Projekt 1 – interfejs wywołań systemowych Linuksa. Grupa: PS8 Imię i nazwisko: Jakub Matusiewicz, Jakub Borkowski	Prowadzący: mgr inż. Tomasz Kuczyński

1 Cel projektu

Celem projektu było stworzenie demona poszukującego plików, który w określonych odstępach czasu rekurencyjnie skanuje system plików w poszukiwaniu ścieżek dopasowanych do zadanych wzorców. Oprócz podstawowej funkcjonalności przeszukiwania i omijania plików bez uprawnień, celem było zaimplementowanie:

- **Współbieżności:** wykorzystania procesów potomnych (jeden proces roboczy na każdy podany argument) oraz procesu nadzorczego do zarządzania skanowaniem.
- **Komunikacji asynchronicznej:** obsługi sygnałów systemowych POSIX (**SIGUSR1**, **SIGUSR2**) do natychmiastowego wybudzania demona, restartowania lub przerywania trwającego przeszukiwania.
- **Szczegółowego raportowania:** zapisywania wyników wyszukiwania oraz opcjonalnych, wyczerpujących informacji o stanie procesu (tryb *verbose*) do logu systemowego (**syslog**).

2 Treść zadania

Temat 2 - Demon(y) poszukujący plików

[20p] Demon(y) poszukujący plików. Program otrzymuje listę argumentów, z których każde jest fragmentem nazwy pliku. Program staje się demonem. Co kilkadziesiąt sekund (domyślny czas można zmienić za pomocą opcjonalnego parametru wiersza poleceń), rekurencyjnie skanuje system plików poszukując plików lub katalogów w których występuje zadany fragment nazwy. Pliki, do których demon nie ma praw dostępu są pomijane. Również takie katalogi są pomijane, dodatkowo katalog bez prawa odczytu i wykonania nie jest brany pod uwagę na etapie przeszukania rekurencyjnego. Nazwy odnalezionych plików lub katalogów są przekazywane zapisywane do logu systemowego (**syslog**). Umieszczone są w nim następujące informacje: pełna data, pełna ścieżka pliku, poszukiwany wzorzec. Odebranie sygnału **SIGUSR1** powoduje natychmiastowe

rozpoczęcie skanowania, gdy demon śpi. Dodatkowa opcja **-v** powoduje przesyłanie do logu wyczerpującej informacji o każdej z następujących czynności demona: a) uśpienie, b) obudzenie się, c) odbiór sygnału, d) porównanie nazwy pliku ze ścieżką.

Dodatkowo:

- a) **[6p]** Odebranie **SIGUSR1**, w chwili gdy trwa przeszukiwanie powoduje natychmiastowy restart przeszukiwania. Odebranie **SIGUSR2**, w chwili gdy trwa przeszukiwanie powoduje natychmiastowe zakończenie przeszukiwania i ponowne uśpienie demona.
- b) **[8p]** Przeszukiwanie jest prowadzone wspólnie przez liczbę procesów równą liczbie argumentów. Dodatkowo występuje proces nadzorczy (śpi w oczekiwaniu na sygnały lub na zakończenie któregoś z procesów przeszukiwanych). Wysłanie procesowi nadzorcemu sygnału **SIGUSR1** albo **SIGUSR2** powoduje jego przekazania wszystkim procesom potomnym.

3 Opis poszczególnych modułów programu

Architektura projektu została podzielona na cztery główne, logiczne moduły funkcjonalne. Taki podział zapewnia czytelność i pozwala na separację mechanizmów systemowych (takich jak obsługa sygnałów i procesów) od logiki biznesowej (skanowanie plików).

3.1 Moduł Inicjalizacji, Konfiguracji i Głównej Pętli (**main**, **args**, **daemonize**, **state**)

Moduł ten odpowiada za punkt wejścia do programu, wczytanie konfiguracji oraz transformację procesu w demona systemowego.

- **Opis działania:** Moduł parsuje argumenty z wiersza poleceń (**args.c**), w tym wzorce poszukiwań, opcjonalny czas uśpienia i flagę **-v** (*verbose*). Następnie wykonuje procedurę odpięcia programu od terminala (**daemonize.c**), powołując do życia demona. W pliku **state.h** zdefiniowano współdzielone struktury konfiguracyjne i stałe czasowe (*runtime*), z których korzysta główna pętla w **main.c**.
- **Kluczowe zmienne globalne:** Struktura konfiguracyjna przechowująca listę argumentów (wzorców) do wyszukiwania, czas uśpienia oraz flagę trybu *verbose*.

3.2 Moduł Nadzorcy i Współbieżności (**supervisor**, **proc_table**, **worker**)

Moduł realizujący wymóg współbieżnego przeszukiwania na podstawie procesów potomnych oraz realizujący logikę procesu nadzorczego (*supervisor*).

- **Opis działania:** Plik `supervisor.c` implementuje orkiestrację procesu nadzorczego. Na podstawie ilości argumentów, tworzone są procesy robocze, czyli workery, gdzie ich implementacja jest w `worker.c`, gdzie każdy odpowiada za jeden cykl skanowania dla konkretnego wzorca. Stan procesów (cykl życia, PID) jest rejestrowany i utrzymywany w strukturach modułu `proc_table.c`, co ułatwia zarządzanie i ewentualne zabijanie/restartowanie workerów.
- **Kluczowe funkcje:** Funkcje monitorujące procesy potomne (`wait/waitpid`) oraz rozsyłające z procesu nadzorczego do workerów otrzymane sygnały (`SIGUSR1/SIGUSR2`).

3.3 Moduł Skanowania Systemu Plików i Raportowania (`scanner`, `match`, `logger`)

Moduł realizujący główną logikę operacji na plikach (I/O) i generowania logów.

- **Opis działania:** W module `scanner.c` zaimplementowano rekurencyjny bezpieczny traversal po systemie plików z uwzględnieniem sprawdzania praw dostępu do katalogów i plików. `match.c` służy jako moduł pomocniczy weryfikujący dopasowanie fragmentu nazwy pliku do zadanych wzorców z użyciem np. `strstr`. Wyniki tych poszukiwań są wysyłane do systemowego logu przez specjalnie przygotowane funkcje z `logger.c`, które filtrują logi na standardowe oraz te dla opcji wyczerpującej (`-v`).

3.4 Moduł Sygnałów i Usypiania (`signals`, `sleep_control`)

Moduł odpowiadający za komunikację asynchroniczną i sterowanie czasem wykonania programu za pomocą sygnałów POSIX.

- **Opis działania:** `signals.c` konfiguruje handlers dla sygnałów `SIGUSR1` (budzenie/restart skanowania) i `SIGUSR2` (przerwanie skanowania i uśpienie). Zmieniają one globalne flagi kontrolne. Z kolei `sleep_control.c` używa tych flag do zarządzania interwałami snu nadzorcy i jego przedwczesnym wybudzaniem.
- **Kluczowe zmienne globalne:** Atomowe flagi systemowe (`volatile sig_atomic_t`), na których opiera się asynchroniczne przerywanie funkcji np. `sleep()`, powiadamiające demona, czy ma natychmiast przestać szukać czy zrestartować skanowanie.

4 Zrzuty ekranu działającego programu (demonstracja)

W celu zaprezentowania poprawności działania programu załączono następujące scenariusze:

1. Standardowe uruchomienie z wieloma wzorcami - widok współbieżnego dopasowania plików do poszczególnych argumentów w `/var/log/syslog`.
2. Przebieg z flagą `-v` pokazujący zdarzenia: uśpienie, wybudzenie, sygnały i porównywanie ścieżek.
3. Obsługa sygnału `SIGUSR1` wysłanego do demona w trakcie snu (natychmiastowe obudzenie).
4. Obsługa sygnałów w trakcie pracy - symulacja wysłania `SIGUSR1` do pracującego demona w celu zrestartowania pętli, a następnie `SIGUSR2` w celu przzerwania skanu i uśpienia całego procesu.

5 Podział pracy

- **Jakub Borkowski:** Implementacja odpięcia procesu (`daemonize`), parsowania argumentów (`args`), procedury bezpiecznego i rekurencyjnego skanowania drzewa katalogów (`scanner`, `match`) oraz logowania do systemowego dziennika (`logger`).
- **Jakub Matusiewicz:** Pisanie sprawozdania, opracowanie obsługi sygnałów `SIGUSR1/SIGUSR2` i flag współdzielonych (`signals`), implementacja procesu nadzorczego i tabeli procesów do zarządzania potomkami (`supervisor`, `proc_table`, `worker`) oraz kontrola wybudzania/usypiania głównej pętli (`sleep_control`).